

MINI PROJECT REPORT

LIBRARY MANAGEMENT SYSTEM

USING PYTHON AND SQLITE

VITyarthi – Build Your Own Project

A modular, database-backed application for managing books, members, circulation, and reports

Field	Details
Student Name	Ananya Monga
Registration Number	26BAI10986
Course / Subject	B.Tech. Computer Science and Engineering (Artificial Intelligence and Machine Learning)
Faculty Name	Dr Rizwan
School / Programme	School of Computing Science & Artificial Intelligence (SCAI)
Submission Date	29/9/2026

Submitted in partial fulfilment of the requirements for the VITyarthi project evaluation.

Abstract

The Library Management System is a small Python application that supports the day-to-day record keeping of a library. It maintains a catalogue of books, member details, and book-loan transactions in an SQLite database. The application provides book and member CRUD operations, book issue and return workflows, inventory updates, active-loan and overdue reports, and a dashboard summary. The project demonstrates modular programming, relational database design, input validation, transaction-based updates, and unit testing. It is intended as a single-user, command-line mini-project for learning and demonstration.

Table of Contents

- 1. Introduction
- 2. Problem Statement
- 3. Objectives
- 4. Scope of the Project
- 5. Functional Requirements
- 6. Non-functional Requirements
- 7. System Architecture
- 8. Design Diagrams
- 9. Database / Storage Design
- 10. Design Decisions and Rationale
- 11. Implementation Details
- 12. Screenshots / Results
- 13. Testing Approach
- 14. Challenges Faced
- 15. Learnings and Key Takeaways
- 16. Future Enhancements
- 17. References

1. Introduction

Libraries need a reliable way to maintain book inventories, member records, and lending activity. When these records are maintained manually, it can become difficult to check availability, track due dates, and maintain consistent records. The Library Management System provides a simple local application to organize these activities through a menu-driven interface.

The application is implemented in Python and uses SQLite for persistent storage. Its code is divided into database, service, and command-line interface modules to keep responsibilities clear and make the project easier to understand and maintain.

2. Problem Statement

Manual library record management can lead to inconsistent inventory counts, duplicated or incomplete records, and difficulty identifying active or overdue loans. A compact system is required to store book and member information, record issue and return transactions, and provide basic reports while enforcing essential data-validation rules.

3. Objectives

- To create and maintain a searchable book catalogue.
- To register and maintain library member records.
- To issue available books to registered members and record due dates.
- To process returns and update available-copy counts consistently.
- To provide active-loan, overdue-loan, and dashboard reports.
- To demonstrate Python modularity, SQLite storage, validation, and unit testing.

4. Scope of the Project

Included: book CRUD and search; member CRUD and search; issuing and returning books; due-date tracking; active and overdue loan reports; dashboard counts; SQLite persistence; validation and unit tests.

Out of scope: multi-user authentication, online access, fine payment, barcode scanning, email notifications, multi-branch operation, and a graphical/web interface. These can be considered as future enhancements.

5. Functional Requirements

Requirement	Description
FR1 – Book management	Add, search, update, and delete book records. Search supports title, author, and ISBN.
FR2 – Member management	Register, search, update, and delete member records using name, email, and phone.
FR3 – Issue book	Issue a book only when the book and member exist and at least one copy is available.
FR4 – Return book	Mark an active loan as returned, record the return date, and restore one available copy.
FR5 – Loan reports	Display active loans and identify loans whose due date has passed and which remain unreturned.

FR6 – Dashboard	Display title count, total copies, available copies, member count, active loans, and overdue count.
FR7 – Validation	Reject missing required fields, invalid copy counts, invalid email formats, duplicate unique fields, and invalid circulation actions.

6. Non-functional Requirements

Requirement	How it is addressed
Usability	A menu-driven CLI provides clear prompts, confirmations, and error messages.
Reliability	SQLite transactions are used for database operations; foreign-key constraints protect relationships.
Data integrity	Checks and validation prevent negative inventory, invalid copy totals, and duplicate ISBN/email values.
Maintainability	Database access, business rules, and user interaction are separated into modules.
Performance	SQLite and an index on loan status are appropriate for a small local dataset.
Portability	The project uses Python's standard library and does not require third-party runtime packages.
Error handling	Expected validation errors are caught and presented to the user rather than terminating the menu.

7. System Architecture

The application follows a simple layered architecture. The user interacts with the CLI. The CLI calls the service layer, which applies business rules and accesses the SQLite database through the database module.

```

flowchart TD
    U[Library Staff / Student] --> CLI[CLI: library/cli.py]
    CLI --> S[Service Layer: library/services.py]
    S --> DB[(SQLite Database)]
    DB --> B[Books]
    DB --> M[Members]
    DB --> L[Loans]

```

Component responsibilities

- `main.py`: initializes the database and starts the menu.
- `library/database.py`: creates the SQLite connection and database schema.
- `library/services.py`: implements validation, CRUD, circulation, and reporting logic.
- `library/cli.py`: receives user input, calls services, and displays results.
- `tests/test_services.py`: contains automated tests for core service behavior.

8. Design Diagrams

8.1 Process workflow

```

flowchart TD
    A([Start]) --> B[Initialize database]
    B --> C[Display menu]
    C --> D{Choose operation}
    D -->|Book/member CRUD| E[Validate and save]
    D -->|Issue book| F{Book available and member valid?}
    F -->|Yes| G[Create loan and reduce stock]
    F -->|No| H[Display error]
    D -->|Return book| I{Loan active?}
    I -->|Yes| J[Mark returned and increase stock]
    I -->|No| H
    E --> C
    G --> C
    J --> C
    H --> C
    D -->|Exit| K([End])
  
```

8.2 Use-case diagram

```

flowchart LR
    Staff([Library Staff]) --> B((Manage Books))
    Staff --> M((Manage Members))
    Staff --> I((Issue Book))
    Staff --> R((Return Book))
    Staff --> V((View Reports))
  
```

8.3 Sequence diagram: issuing a book

```

sequenceDiagram
    actor Staff
    participant CLI
    participant Service
    participant DB as SQLite
    Staff->>CLI: Enter book ID and member ID
    CLI->>Service: issue_book(book_id, member_id)
    Service->>DB: Validate book, member, and stock
    DB-->>Service: Return records
    Service->>DB: Insert loan and decrement stock
    DB-->>Service: Commit transaction
    Service-->>CLI: Return loan ID
    CLI-->>Staff: Display confirmation
  
```

8.4 Class / component diagram

```

classDiagram
    class Database {
        +connect()
        +initialize()
    }
    class LibraryService {
        +add_book()
        +update_book()
        +delete_book()
        +add_member()
        +issue_book()
        +return_book()
        +list_loans()
        +overdue_loans()
        +dashboard()
    }
  
```

```

}
class CLI {
    +run()
    +show()
}
CLI --> LibraryService
LibraryService --> Database

```

9. Database / Storage Design

SQLite is used as a local relational database. The schema contains three tables: books, members, and loans. A loan references one book and one member. Returned loans are retained to preserve transaction history.

```

erDiagram
    BOOKS ||--o{ LOANS : has
    MEMBERS ||--o{ LOANS : borrows
    BOOKS {
        int id PK
        string title
        string author
        string isbn UK
        int total_copies
        int available_copies
    }
    MEMBERS {
        int id PK
        string name
        string email UK
        string phone
    }
    LOANS {
        int id PK
        int book_id FK
        int member_id FK
        date issue_date
        date due_date
        date return_date
        string status
    }

```

Key schema rules

- Books: primary key id; title and author are required; ISBN is unique when supplied; copy counts must be non-negative and available copies cannot exceed total copies.
- Members: primary key id; name and email are required; email is unique.
- Loans: primary key id; book_id and member_id are foreign keys; status is either ISSUED or RETURNED.

10. Design Decisions and Rationale

- SQLite: lightweight, file-based, and supported by Python's standard library.
- Layered modular design: separates interface, business rules, and persistence for easier maintenance.
- Loan history: records are retained after return rather than deleted.
- Inventory consistency: issuing decreases available copies; returning increases them. Both actions are performed with their associated loan updates.
- Command-line interface: keeps the project small, easy to run, and suitable for an introductory programming project.

11. Implementation Details

The implementation uses Python 3.10+ and the built-in sqlite3 module. No third-party packages are required. On startup, main.py initializes the schema and launches the menu. The application supports book/member CRUD, circulation, reports, and dashboard statistics.

To run the application on a system with Python 3.10 installed:

```
py -3.10 main.py
```

To run the automated tests:

```
py -3.10 -m unittest discover -s tests -v
```

The database file library.db is generated automatically in the project directory. It is excluded from version control through .gitignore so that local data is not accidentally committed.

12. Screenshots / Results

Insert actual screenshots after running the application. The following screenshots are recommended for the final submission:

- Main menu displayed in the terminal.
- Book catalogue showing at least one book record.
- Member listing showing a registered member.
- Successful issue operation showing the loan ID.
- Return operation and updated available-copy count.
- Dashboard or overdue report.

Screenshot placeholder: Main menu

```

=== LIBRARY MANAGEMENT ===
1. Dashboard
2. List/search books
3. Add book
4. Update book
5. Delete book
6. List members
7. Add member
8. Update member
9. Delete member
10. Issue book
11. Return book
12. Active loans
13. Overdue report
0. Exit

Choose: |

```

```
=== LIBRARY MANAGEMENT ===
```

1. Dashboard
2. List/search books
3. Add book
4. Update book
5. Delete book
6. List members
7. Add member
8. Update member
9. Delete member
10. Issue book
11. Return book
12. Active loans
13. Overdue report
0. Exit

```
Choose: 3
```

```
Title: C++
```

```
Author: S.Bala
```

```
ISBN (optional): 12323
```

```
Copies: 3|
```

```
Choose: 6
```

```
Search name/email (blank = all):
```

```
id=1 | name=Ananya | email=anu@gmail.com | phone=121212
```

```
=== LIBRARY MANAGEMENT ===
```

1. Dashboard
2. List/search books
3. Add book
4. Update book
5. Delete book
6. List members
7. Add member
8. Update member
9. Delete member
10. Issue book
11. Return book
12. Active loans
13. Overdue report
0. Exit

```
Choose: 7
```

```
Name: Ananya
```

```
Email: anu@gmail.com
```

```
Phone: 121212
```

```
Created member ID: 1
```

```
=== LIBRARY MANAGEMENT ===
```

1. Dashboard
2. List/search books
3. Add book
4. Update book
5. Delete book
6. List members
7. Add member
8. Update member
9. Delete member
10. Issue book
11. Return book
12. Active loans
13. Overdue report
0. Exit

13. Testing Approach

Unit tests are implemented using Python's built-in unittest framework. The tests use a temporary database so that test records do not alter the user's normal library database.

Test case	Expected behavior
Issue and return inventory	Issuing the only copy reduces availability to zero;

	returning it restores availability to one.
Unavailable book	A second issue attempt fails when no copies remain available.
Required fields	Adding a book without a title raises a validation error.

The project test run completed successfully: 3 tests ran and passed. This result reflects the included automated tests; additional manual testing should be performed before final submission.

14. Challenges Faced

- Maintaining correct inventory counts across issue and return operations.
- Preventing invalid operations, such as issuing a book with no available copies.
- Maintaining referential integrity between books, members, and loans.
- Designing a compact interface while keeping the code modular.

These challenges are addressed through validation in the service layer, SQLite constraints, and automated tests.

15. Learnings and Key Takeaways

- Using Python modules to separate application responsibilities.
- Connecting Python applications to SQLite databases with `sqlite3`.
- Designing relational tables, primary keys, foreign keys, and constraints.
- Implementing CRUD operations and parameterized SQL queries.
- Applying validation and exception handling.
- Writing repeatable unit tests with `unittest`.
- Using Git and GitHub to maintain and submit a project.

16. Future Enhancements

- User login and role-based access for librarians and members.
- Fine calculation for overdue books.
- CSV/Excel export of inventory and circulation reports.
- Barcode or QR-code support for books.
- Graphical user interface or web-based interface.
- Email reminders for approaching due dates.
- Support for multiple library branches and concurrent users.

17. References

1. Python Software Foundation, Python 3 Documentation, <https://docs.python.org/3/>
2. SQLite, SQLite Documentation, <https://www.sqlite.org/docs.html>
3. VITyarthi, Build Your Own Project – General Project Instructions & Submission Guidelines, user-provided course document.

— End of Report —